SIMATS ENGINEERING

ASSESSMENT TOOL 1-Pseudocode Writing Task (Algorithm Design)

COURSE NAME: Data Structures COURSE CODE: CSA03

COURSE OUTCOME COVERED

CO5: Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem. (Bloom Level -BL4, BL6)

Assessment Tool Weightage: 35%

QUESTIONS: EACH QUESTION CARRIES: 10 MARKS DURATION : 1 HOUR

25. **Second Best Spanning Tree:**

Design pseudocode to find the "Second Best" MST.

- *Hint:* Find the MST first, then for every edge in the MST, try removing it and finding the next best edge to replace it.

SECOND_BEST_MST(Graph G)

1. MST ← KRUSKAL(G)
2. bestCost ← COST(MST)
3. secondBestCost ← ∞
4. secondBestTree ← NULL

5. FOR each edge e in MST DO

6. Remove edge e from MST

7. components ← Find connected components of MST

8. replacementEdge ← NULL
9. minimumWeight ← ∞

10. FOR each edge x in G DO

11. IF x is NOT in MST AND

```
       x connects two different components THEN
         IF weight(x) < minimumWeight THEN
           minimumWeight ← weight(x)
           replacementEdge ← x
         END IF
       END IF
     END FOR
     IF replacementEdge ≠ NULL THEN
       newTree ← MST + replacementEdge
       newCost ← COST(newTree)
       IF newCost > bestCost AND
       newCost < secondBestCost THEN
         secondBestCost ← newCost
         secondBestTree ← newTree
       END IF
     END IF
     Restore edge e to MST
   END FOR
   RETURN secondBestTree, secondBestCost
END
```

Output

```
Enter number of vertices: 4

5

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4
Enter number of edges: Enter edges (source destination weight):

MST Cost = 19
Second Best MST Cost = 20
```

Code of Conduct:

I JAYANTH B (192511375) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	**20%**	**Identifies all inputs, outputs, constraints accurately**	**Minor missing elements**	**Partial understanding**	**Incorrect interpretation**
Algorithm Design	**30%**	**Complete, correct, and logically sound solution**	**Minor logical gaps**	**Partially correct logic**	**Incorrect approach**
Use of Control Structures	**20%**	**Efficient and appropriate use of loops, conditions, functions/modules**	**Minor inefficiencies**	**Limited or basic usage**	**Incorrect or missing constructs**
Pseudocode Structure	**10%**	**Well-structured, clear, consistent format, easy to follow**	**Mostly clear with minor issues**	**Difficult to follow in parts**	**Poorly structured and unclear**
Completeness	**20%**	**Handles all cases; optimized approach**	**Handles most cases; reasonable efficiency**	**Handles basic cases only**	**Incomplete or inefficient**